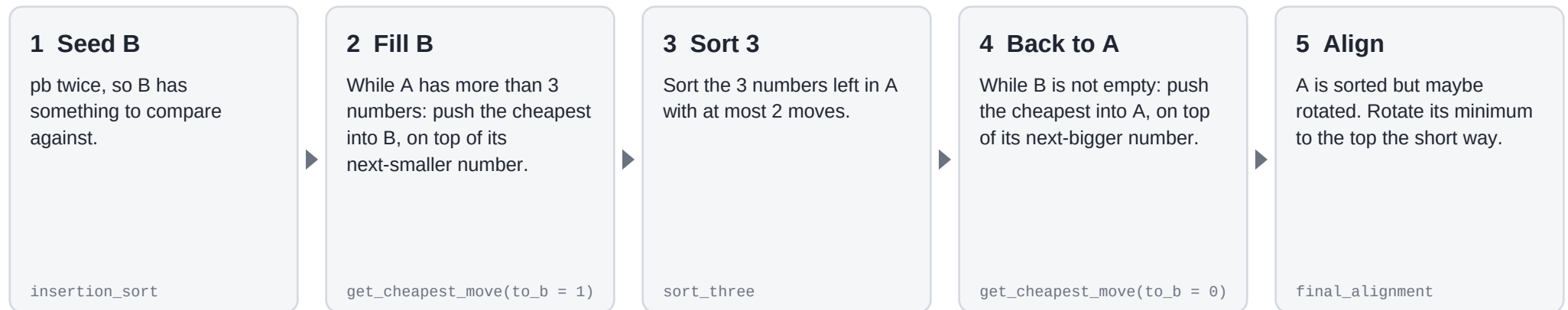


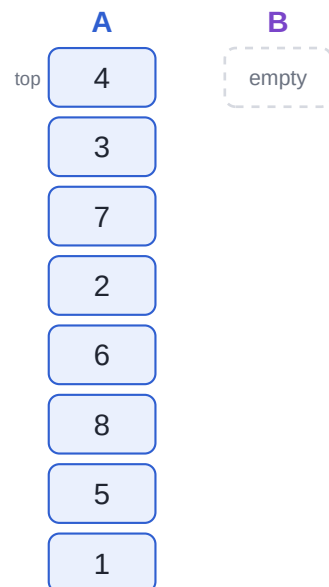
How the simple sort works, step by step

Cheapest insertion (the "Turk" algorithm): `algorithms/simple.c`, `simple_cost.c`, `simple_rotation.c`

The idea: keep **B sorted descending** while filling it, leave 3 numbers in A, then insert everything back into A so that **A stays sorted ascending**. On every push we price all candidates and take the one that needs the fewest rotations.



THE EXAMPLE IN THIS WALKTHROUGH



Input `./push_swap 4 3 7 2 6 8 5 1`. The first argument ends up on top of A.

The code compares each node's `index` (its rank), not its value. With the numbers 1 to 8 the rank is just value minus 1, so the order is the same and the pages show the values.

In total the algorithm needs **16 operations**. Every step on the following pages was produced by a Python replica of the C code whose output was checked against the real `push_swap` binary.

Colours: an **orange** box is the number chosen to move, a **teal** box is its target (the number it will land on).

The two rules behind every push

How a rotation cost is counted, and how two costs combine into one total.

1. Rotation cost is signed

get_rotation_cost counts how many moves bring a node to the top. Top half: rotate up (**ra**), cost is **positive**. Bottom half: rotate down (**rra**) because it is shorter, cost is **negative**.

A (size 6)			
top	9	position 0	0 already on top
	4	position 1	+1 1 x ra
	7	position 2	+2 2 x ra
	1	position 3	+3 3 x ra
	6	position 4	-2 2 x rra
	2	position 5	-1 1 x rra

Rule: if position $\leq$ size / 2 the cost is position, otherwise position - size.

2. Same direction = shared moves

get_total_cost combines the cost in A and the cost in B. If both go the same way, **rr** / **rrr** rotate both stacks in one move.

cost A = +3, cost B = +2

same way: total = $\max(3, 2) = 3$

rr rr ra

cost A = -1, cost B = -2

same way: total = $\max(1, 2) = 2$

rrr rrb

cost A = +2, cost B = -1

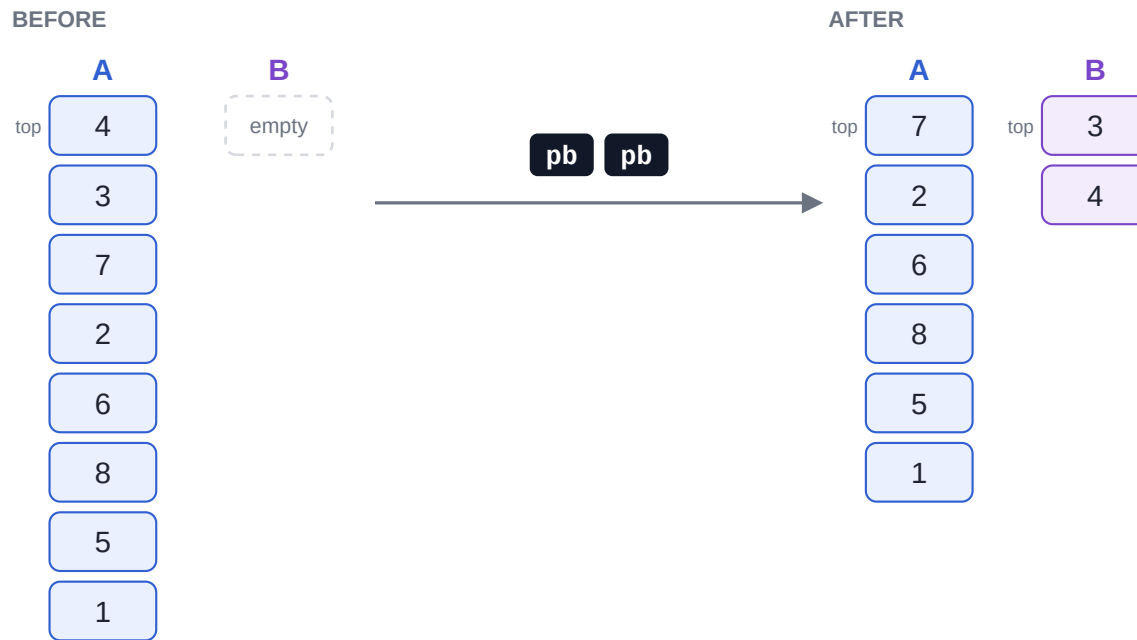
opposite: total = $2 + 1 = 3$

ra ra rrb

apply_move always runs the shared rotations first (rotate_both), then whatever is left on each stack (rotate_single).

Seed B with two numbers

A has more than 3 numbers, so push the top two to B without pricing anything.



insertion_sort does **pb** twice. B now has something to compare against. Its order (3 4) doesn't matter yet, since two numbers are always circularly sorted.

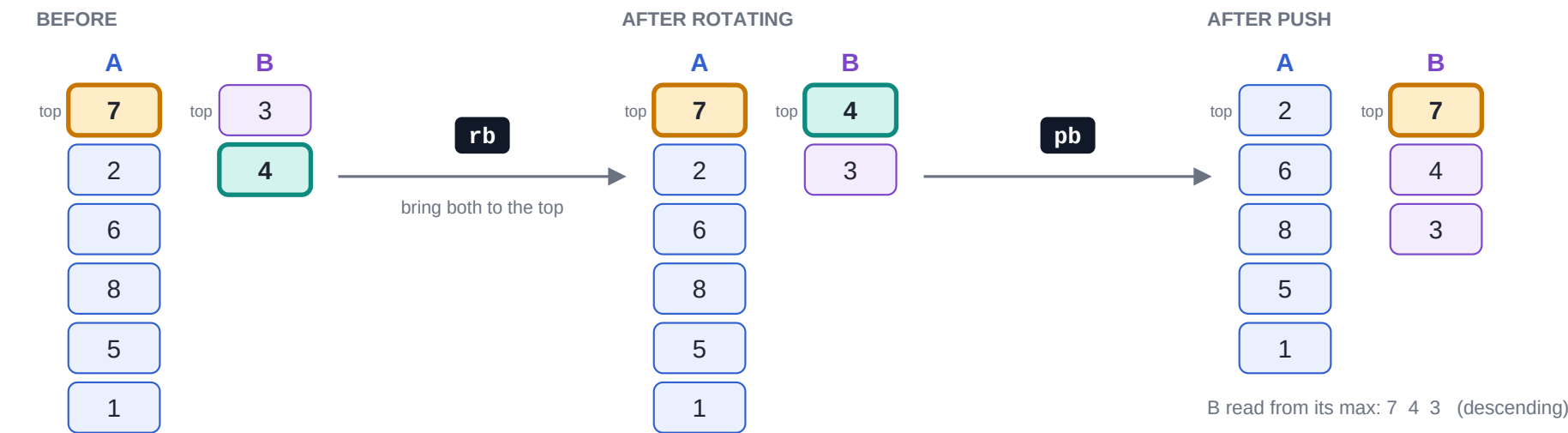
From here on, every push to B keeps B in descending order (read circularly from its maximum).

Push 1 of 3: move 7 from A to B

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in A	target in B	cost A	cost B	total
7	4	0	+1 (1 rb)	1
2	4	+1 (1 ra)	+1 (1 rb)	1
6	4	+2 (2 ra)	+1 (1 rb)	2
8	4	+3 (3 ra)	+1 (1 rb)	3
5	4	-2 (2 rra)	+1 (1 rb)	3
1	4	-1 (1 rra)	+1 (1 rb)	2

Target rule (A to B): the biggest number in B that is *smaller*. B is kept in descending order, so the new number must sit right on top of its next-smaller neighbour.

Cheapest: 7 (total 1; the first lowest total wins ties). The biggest number in B below 7 is 4.

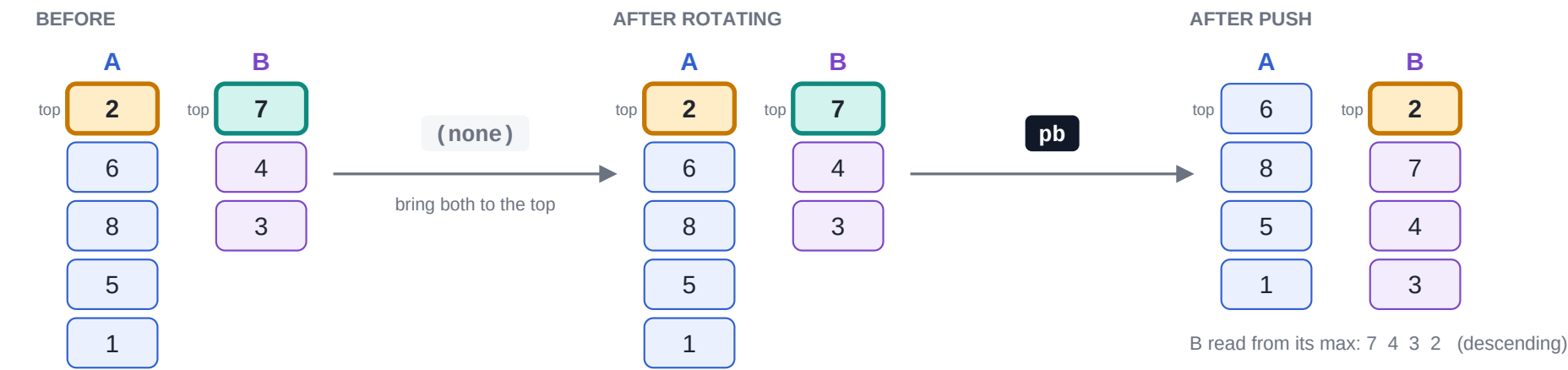
Moves: Only one stack needs to move: total = 1. Run 1x **rb**, then **pb** drops 7 on top of 4.

Push 2 of 3: move 2 from A to B

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in A	target in B	cost A	cost B	total
2	7	0	0	0
6	4	+1 (1 ra)	+1 (1 rb)	1
8	7	+2 (2 ra)	0	2
5	4	-2 (2 rra)	+1 (1 rb)	3
1	7	-1 (1 rra)	0	1

Target rule (A to B): the biggest number in B that is *smaller*. B is kept in descending order, so the new number must sit right on top of its next-smaller neighbour.

Cheapest: 2 (total 0; the first lowest total wins ties). Nothing in B is smaller than 2, so it is a new minimum. Its target falls back to B's maximum **7**: in a circular descending stack the minimum sits just above the maximum.

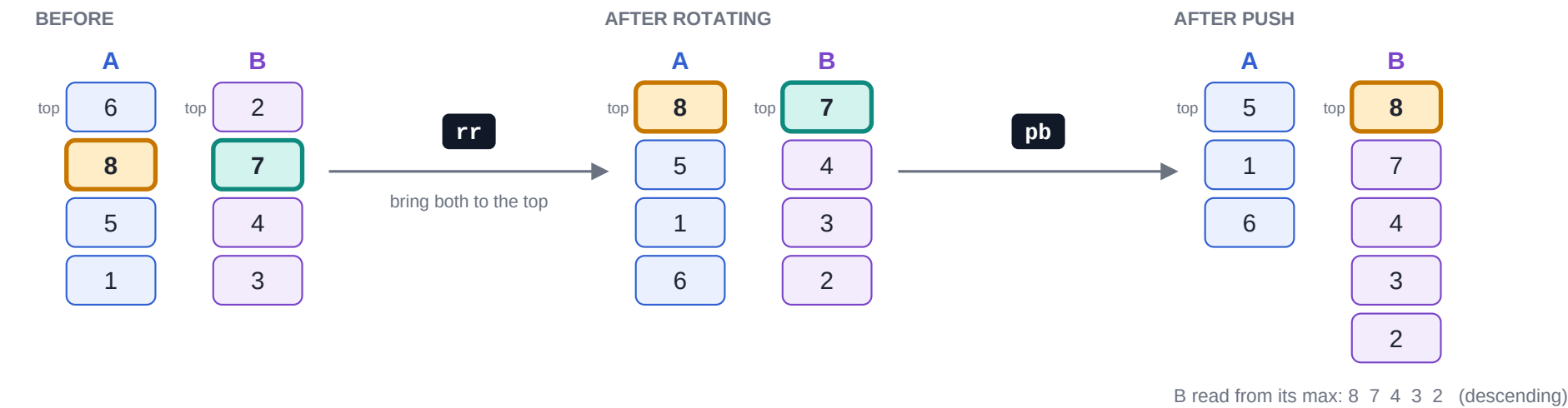
Moves: 2 and 7 are both already on top, so no rotations: **pb** drops 2 straight onto 7.

Push 3 of 3: move 8 from A to B

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in A	target in B	cost A	cost B	total
6	4	0	+2 (2 rb)	2
8	7	+1 (1 ra)	+1 (1 rb)	1
5	4	+2 (2 ra)	+2 (2 rb)	2
1	7	-1 (1 rra)	+1 (1 rb)	2

Target rule (A to B): the biggest number in B that is *smaller*. B is kept in descending order, so the new number must sit right on top of its next-smaller neighbour.

Cheapest: 8 (total 1; the first lowest total wins ties). The biggest number in B below 8 is 7.

Moves: Both costs point the same way, so the rotations are shared: total = max(1, 1) = 1. Run 1x **rr**, then **pb** drops 8 on top of 7.

Sort the 3 numbers left in A

A is down to 3 numbers: `sort_three` sorts them directly.



sort_three first moves the maximum to the bottom (**ra** if it is on top, **rra** if it is in the middle), then swaps the top two if needed. That is at most 2 moves.

Here the maximum 6 is already at the bottom, so only **sa** is needed.

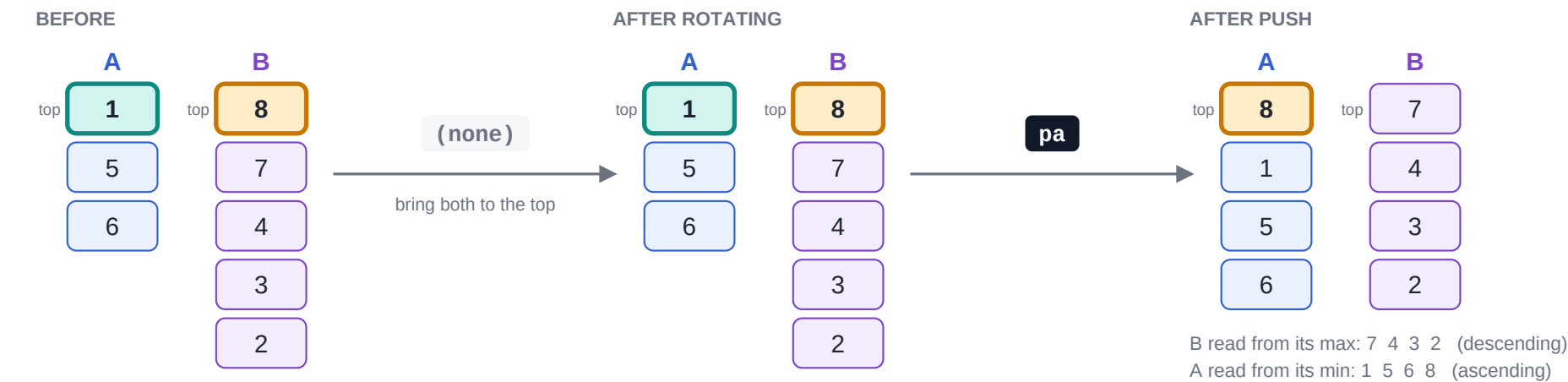
A is now ascending, and every push back from B keeps it that way.

Push 1 of 5: move 8 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
8	1	0	0	0
7	1	0	+1 (1 rb)	1
4	5	+1 (1 ra)	+2 (2 rb)	2
3	5	+1 (1 ra)	-2 (2 rrb)	3
2	5	+1 (1 ra)	-1 (1 rrb)	2

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 8 (total 0; the first lowest total wins ties). Nothing in A is bigger than 8, so it is a new maximum. Its target falls back to A's minimum **1**: in a circular ascending stack the maximum sits just above the minimum.

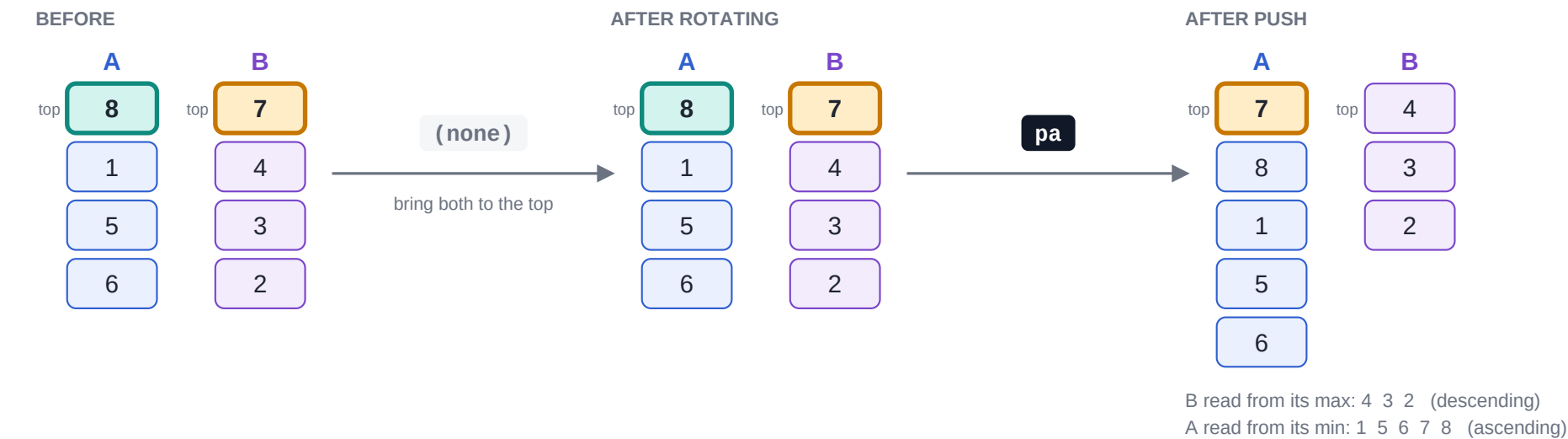
Moves: 8 and 1 are both already on top, so no rotations: **pa** drops 8 straight onto 1.

Push 2 of 5: move 7 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
7	8	0	0	0
4	5	+2 (2 ra)	+1 (1 rb)	2
3	5	+2 (2 ra)	+2 (2 rb)	2
2	5	+2 (2 ra)	-1 (1 rrb)	3

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 7 (total 0; the first lowest total wins ties). The smallest number in A above 7 is 8.

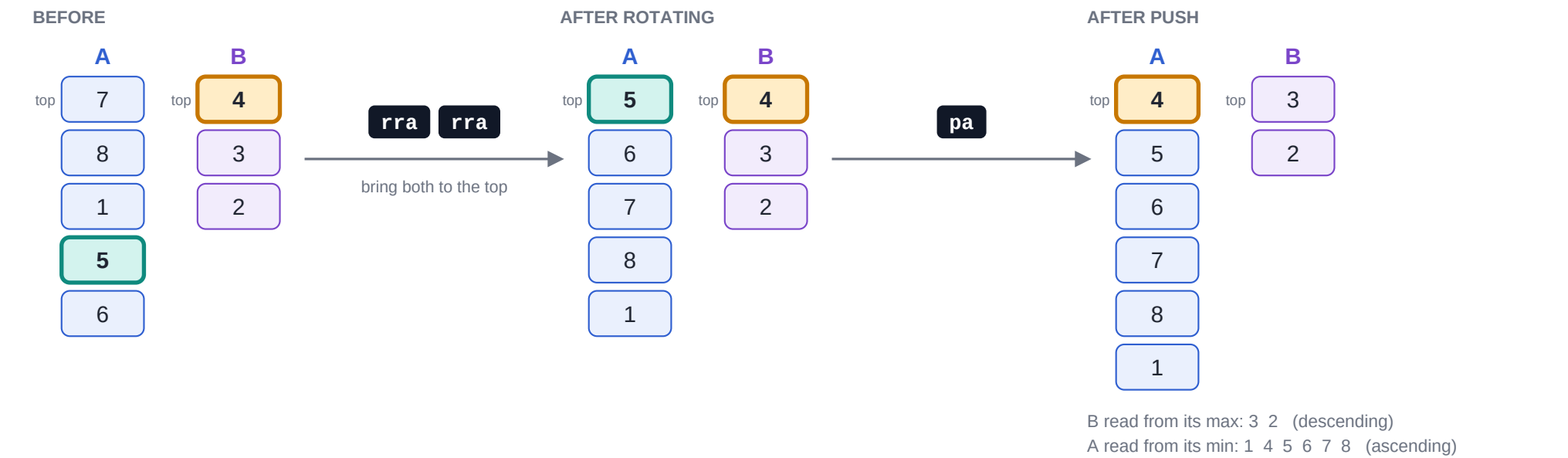
Moves: 7 and 8 are both already on top, so no rotations: **pa** drops 7 straight onto 8.

Push 3 of 5: move 4 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
4	5	-2 (2 rra)	0	2
3	5	-2 (2 rra)	+1 (1 rb)	3
2	5	-2 (2 rra)	-1 (1 rrb)	2

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 4 (total 2; the first lowest total wins ties). The smallest number in A above 4 is 5.

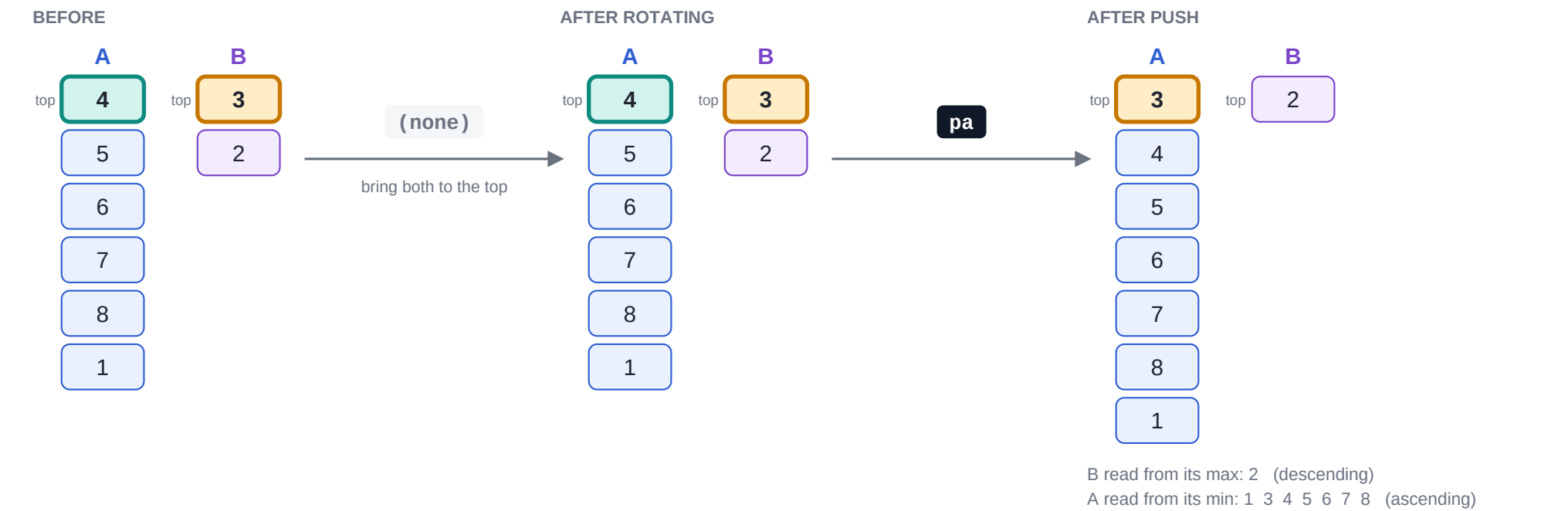
Moves: Only one stack needs to move: total = 2. Run 2x **rra**, then **pa** drops 4 on top of 5.

Push 4 of 5: move 3 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
3	4	0	0	0
2	4	0	+1 (1 rb)	1

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 3 (total 0; the first lowest total wins ties). The smallest number in A above 3 is 4.

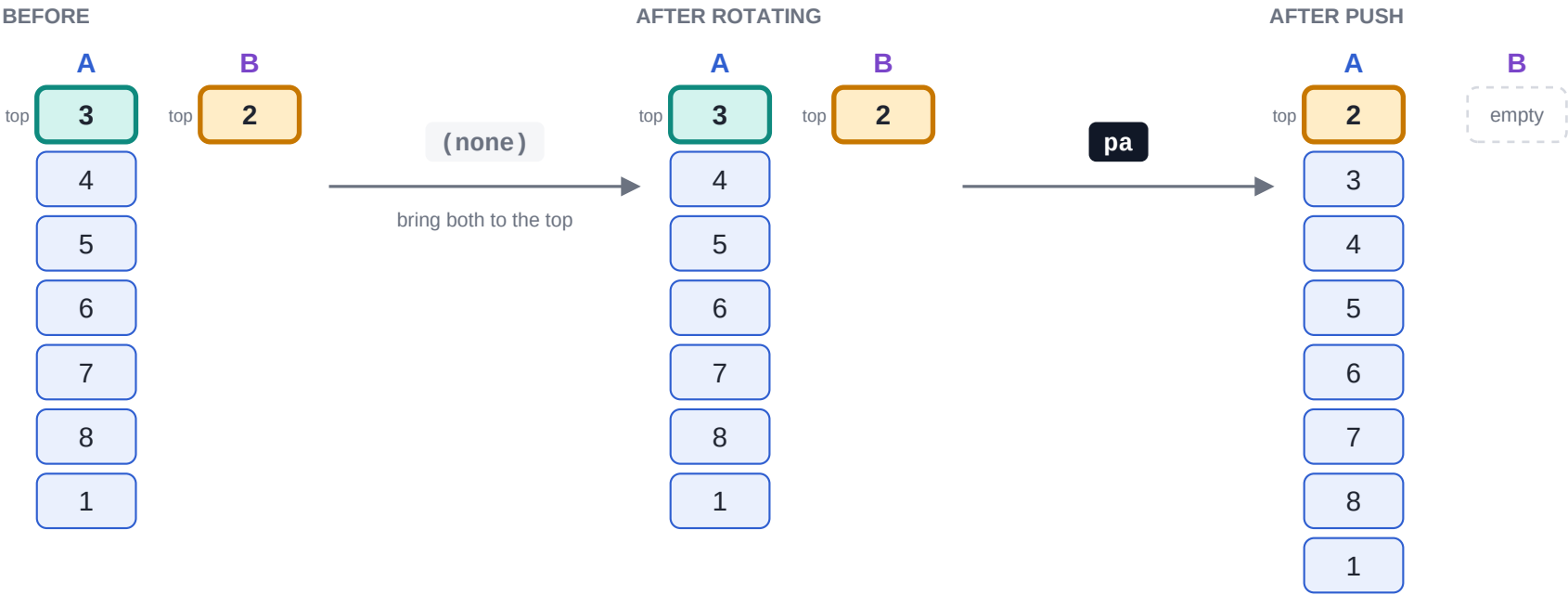
Moves: 3 and 4 are both already on top, so no rotations: **pa** drops 3 straight onto 4.

Push 5 of 5: move 2 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



A read from its min: 1 2 3 4 5 6 7 8 (ascending)

EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
2	3	0	0	0

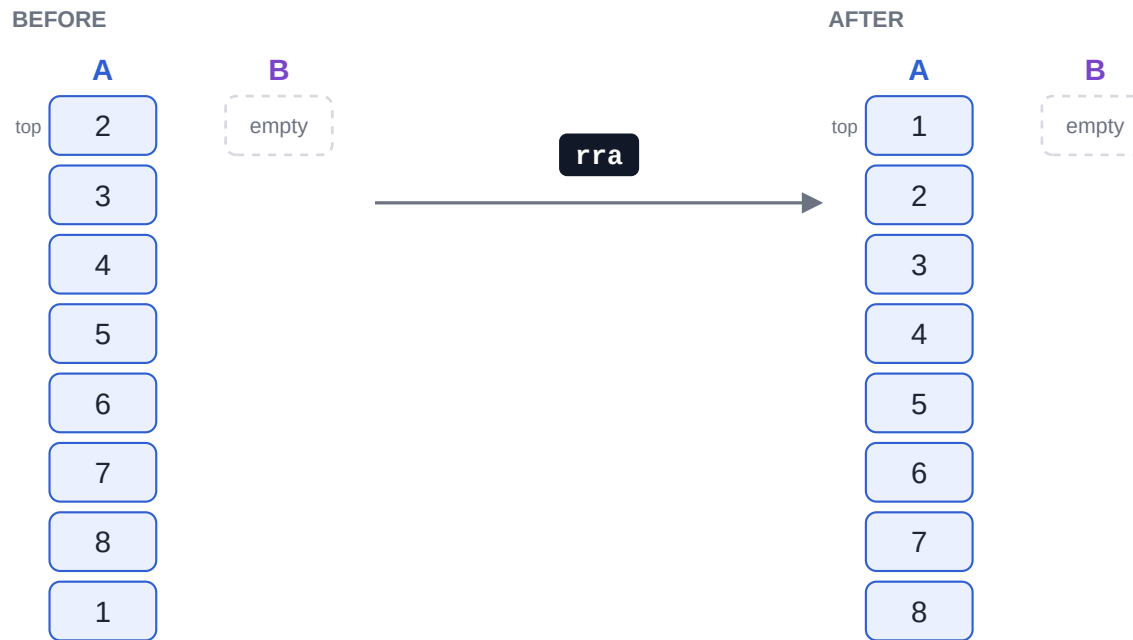
Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 2 (total 0; the first lowest total wins ties). The smallest number in A above 2 is **3**.

Moves: 2 and 3 are both already on top, so no rotations: **pa** drops 2 straight onto 3.

Rotate the minimum to the top

A is sorted in a circle but may start in the wrong place.



A reads 1 2 3 4 5 6 7 8 from its minimum, so it is sorted but rotated. The minimum 1 is at position 7 of 8.

final_alignment reuses the same signed cost: -1, which means 1x **rra**.

Done: sorted in 16 operations

The full operation list, exactly as ./push_swap prints it.

Seed B

2 ops

pb pb

Fill B

5 ops

rb pb pb rr pb

Sort 3

1 ops

sa

Back to A

7 ops

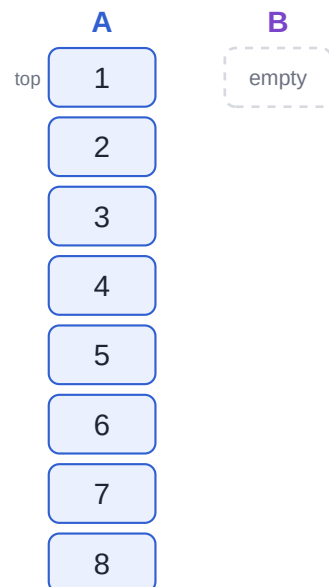
pa pa rra rra pa pa pa

Align

1 ops

rra

FINAL STACKS



Why B descending? On the way back, the number under the top of B is usually the next-smaller one, and its target in A is the number just pushed, which is already on top. That is why phase 4 here is mostly bare **pas**.

Why A ascending? That is the answer. Every insert into A must use the *next-bigger* target, otherwise A ends up out of order.

Why "cheapest"? Always pushing the top number would work too, but pricing every candidate and sharing **rr/rrr** keeps the move count low (about 5,000 moves for 500 numbers).

Cost of the search: each push prices every candidate and walks the other stack to find its target, so the whole sort does $O(n^2)$ work.